

LÓGICA DE PROGRAMAÇÃO

Aula 1 — Variáveis, Condicionais e Laços de Repetição

Nesta aula vamos estudar os três primeiros pilares da lógica de programação: variáveis e tipos de dados, estruturas condicionais e estruturas de repetição (laços). Cada tópico traz o conceito, a lógica por trás dele e exemplos práticos em JavaScript e em C#, para que você perceba que a lógica é a mesma — só a sintaxe muda de uma linguagem para outra.

Tópico 1 — Variáveis e Tipos de Dados

Uma variável é um espaço nomeado na memória do computador usado para guardar um valor que pode mudar ao longo da execução do programa. Pense nela como uma caixa com etiqueta: a etiqueta é o nome da variável, e dentro da caixa fica o valor guardado.

Tipos de dados primitivos mais comuns

- Número (inteiro ou decimal) — ex.: 10, 3.14
- Texto (string) — ex.: "Barbacena"
- Booleano — verdadeiro (true) ou falso (false)

Como declarar uma variável

Em JavaScript usamos `let` (valor que pode mudar) ou `const` (valor fixo). Em C# a linguagem é fortemente tipada: precisamos declarar o tipo do dado (`int`, `string`, `bool`, `double`...) antes do nome da variável.

JavaScript <pre>let nome = "Ana"; let idade = 17; let altura = 1.65; let aprovado = true; console.log(nome, idade);</pre>	C# <pre>string nome = "Ana"; int idade = 17; double altura = 1.65; bool aprovado = true; Console.WriteLine(nome + " " + idade);</pre>
--	--

Ponto de atenção

Em JavaScript o tipo é dinâmico (a mesma variável pode mudar de tipo em tempo de execução). Em C# o tipo é estático: uma vez declarada como `int`, aquela variável nunca vira `string`. Esse é o principal choque cultural entre as duas linguagens.

Tópico 2 — Estruturas Condicionais

Estruturas condicionais permitem que o programa tome decisões: dependendo de uma condição ser verdadeira ou falsa, um bloco de código diferente é executado. É a lógica do 'se isso, então aquilo, senão aquilo outro'.

if / else if / else

A sintaxe do if é praticamente idêntica entre JavaScript e C#, com pequenas diferenças de convenção de nomes (Console.WriteLine em vez de console.log, por exemplo).

JavaScript <pre>let nota = 7; if (nota >= 7) { console.log("Aprovado"); } else if (nota >= 5) { console.log("Recuperação"); } else { console.log("Reprovado"); }</pre>	C# <pre>int nota = 7; if (nota >= 7) { Console.WriteLine("Aprovado"); } else if (nota >= 5) { Console.WriteLine("Recuperação"); } else { Console.WriteLine("Reprovado"); }</pre>
---	---

Operadores lógicos e de comparação

- Comparação: == (igual), != (diferente), >, <, >=, <=
- Lógicos: && (E), || (OU), ! (negação)
- Em JavaScript existe também === (igual estrito, compara valor e tipo) — recomendado no lugar de ==

Tópico 3 — Estruturas de Repetição (Laços)

Laços de repetição servem para executar um bloco de código várias vezes, sem precisar repetir o código manualmente. Usamos quando sabemos o número de repetições (for) ou quando a repetição depende de uma condição (while).

Laço for

JavaScript <pre>for (let i = 1; i <= 5; i++) { console.log("Contagem: " + i); }</pre>	C# <pre>for (int i = 1; i <= 5; i++) { Console.WriteLine("Contagem: " + i); }</pre>
--	--

Laço while

JavaScript

```
let contador = 0;
while (contador < 3) {
  console.log(contador);
  contador++;
}
```

C#

```
int contador = 0;
while (contador < 3) {
  Console.WriteLine(contador);
  contador++;
}
```

Cuidado com laços infinitos

Se a condição do while nunca se tornar falsa (por exemplo, esquecer o contador++), o programa entra em loop infinito e trava. Sempre confira se a variável de controle está sendo atualizada dentro do laço.

Exercícios de Fixação

Resolva os exercícios abaixo em uma das duas linguagens (ou nas duas, para praticar):

- 1. Crie uma variável com o preço de um produto e aplique um desconto de 10% se o preço for maior que R\$ 100.
- 2. Escreva um programa que receba uma nota e informe se o aluno está Aprovado, em Recuperação ou Reprovado.
- 3. Use um laço for para imprimir os números de 1 a 10, e um laço while para imprimir apenas os números pares de 1 a 10.

Na próxima aula veremos os dois pilares seguintes do nosso plano: funções e coleções (arrays/listas).